Wei Xiong - Project Portfolio Page

Overview

Bob is a Java command-line application for university lab technicians to manage inventory, sessions, preparation, contacts, requests, loans, and broken items. My main contributions were the original inventory feature and student request tracking, including stock deduction on fulfillment.

GitHub: Lokwx | Team repository | Code contributed

Summary of Contributions

- **Inventory:** Implemented `Inventory`, `InventoryItem`, and the original `add-i`, `delete-i`, and `list-i` (implementation). This shared stock supports other features. Dayna later added validation, case-insensitive matching, ordering, and category cleanup (changes).
- **Requests:** Implemented deadline ordering, live availability, and four request commands (PR #45), then cross-category fulfillment (PR #72). To keep stock and status consistent, fulfillment checks total stock, deducts in display order, then updates status. Insufficient stock changes neither. Gavin later strengthened final-status protection and completed-request display (PR #73).
- **Tests:** Added request exceptions, assertions, logging, overflow-safe totals, and clock injection for deterministic date tests. Wrote inventory and request tests, including insufficient stock, cross-category deduction, and repeated/reversed fulfillment (regressions). Teammates added preparation tests in the shared availability test class.
- **User Guide:** Wrote request instructions and expanded inventory examples and validation feedback (examples).
- **Developer Guide:** Explained inventory addition with class/sequence diagrams, and requests, sorting, availability, fulfillment, and alternatives (inventory diagrams).
- **Team tasks:** Corrected UML and guide inconsistencies, supplied diagram sources, and aligned guides with persistence/history (PR #78); Dayna implemented those storage features. Added standards and Checkstyle configuration.
- **Reviews:** Identified an invalid stock-changing example in the preparation DG (#56), broken-item exclusions in lookup claims (#58), and unclosed RepoSense author tags (#57).

Contributions to the User Guide - Selected Extracts

Abridged from my inventory and request instructions, including later team refinements.

Register stock and fulfill a request

Start Bob in a new, empty working directory and enter your name. These examples assume the computer's date is no later than 8 October 2099.

Formats: `add-i n/NAME c/CATEGORY [q/QUANTITY]` and `add-r n/STUDENT_NAME i/ITEM q/QUANTITY d/DEADLINE`. Supply prefixes in the order shown. Names and categories cannot be blank. Quantities must be positive whole numbers up to 2,147,483,647, entered without commas. Only inventory additions default to one when quantity is omitted. Duplicate inventory names within a category are rejected ignoring case; adding does not restock an entry.

```
add-i n/Wire c/Components q/15
add-i n/Wire c/Spares q/5
add-r n/Alice i/Wire q/20 d/8 October 2099
list-r
mark-r 1 prepared
mark-r 1 fulfilled
list-r
```

The request starts pending. Adding it and marking it prepared leave all 20 Wire available. Fulfillment deducts 15 from Components and five from Spares. The final list shows `[Fulfilled]` and `Available: 0`, without a shortage warning.

Deadline and index rules: Use a real date today or later in `d MMMM yyyy` format, with a full English month name and an initial capital. Requests are sorted by deadline, then student and item names ignoring case. Run `list-r` after additions or deletions before using an index.

Stock and status rules: `mark-r INDEX STATUS` accepts `prepared` or `fulfilled`, ignoring case. Requests combine matching item names across categories. Adding or preparing does not reserve stock; each request checks current availability independently. Insufficient stock rejects fulfillment without changing stock or status.

Fulfillment is final: It cannot be repeated or reversed. Deleting a fulfilled request does not restore stock. Fulfillment does not create a loan; use `add-l` for a loan to avoid deducting the same units twice.

Contributions to the Developer Guide - Selected Extracts

Abridged from my inventory and request documentation, including team refinements credited in the summary.

Inventory addition: responsibilities and relationships

Source: Inventory management.

`Parser` validates prefix order, trims values, parses quantity, and creates an `InventoryItem`. The item validates its fields independently. `Inventory` resolves the category, rejects duplicate names, stores the supplied item reference, and prints confirmation.

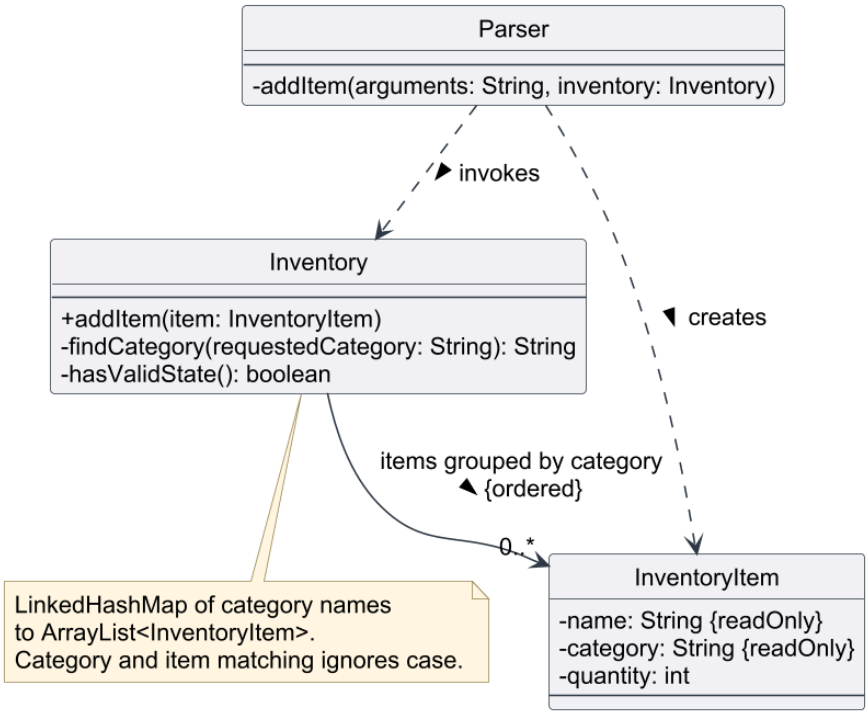


Figure 1: Parser uses Inventory and creates InventoryItem objects. Inventory retains item references grouped by category; the association does not promise exclusive ownership.

The current `LinkedHashMap<String, ArrayList<InventoryItem>>` preserves category and item insertion order. Case-insensitive resolution preserves stored spelling. These ordering and matching refinements include Dayna's inventory changes. Other features query the same inventory for usable stock.

Inventory addition: ordering and validation

For `add-i n/Screws c/Fasteners q/100`, the parser validates the input and creates an item. Inventory creates the category if necessary, checks for duplicates, then appends the item. A duplicate such as `add-i n/screws c/FASTENERS q/20` is rejected without changing the existing 100 units.

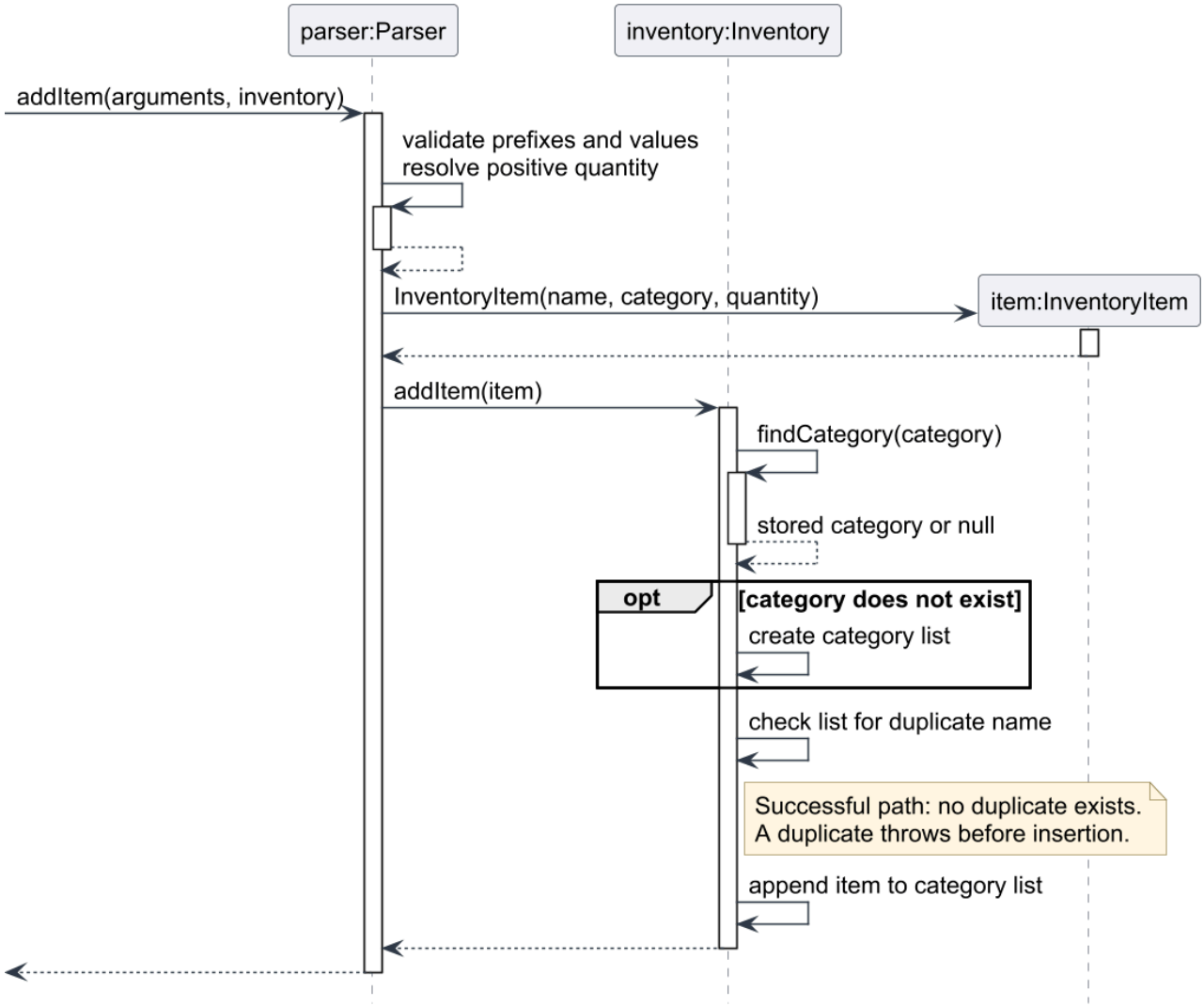


Figure 2: Successful addition after command dispatch. Construction validates the item, and duplicate checking precedes insertion. Routine getters, assertions, logging, and console output are omitted.

Parser validation provides command-specific feedback; constructor checks protect direct callers. Assertions check internal invariants, including positive stored quantities and nonempty categories. They detect programming errors and do not replace input validation. Inventory diagnostics use `FINE` without changing normal output.

Diagram sources: class diagram and addition sequence.

Requests: sorting, availability, and fulfillment

Sources: Request tracking and fulfillment design.

`RequestManager` stores an `ArrayList<Request>` and sorts after addition by deadline, then student and item names ignoring case. Stable sorting preserves insertion order for equal keys. Quantity and status are not comparison keys, so marking does not require sorting. Sorting only for display would require translating action indices.

`Inventory#getTotalAvailableQuantity` matches full item names ignoring case across categories. It sums in a `long` and caps the result at `Integer.MAX_VALUE`. Listing reads live stock, reflecting loans and broken-item reports without a cached total. Each request is checked independently; availability is not a reservation.

Fulfillment flow:

1. `Parser` validates the index and status and passes the shared inventory to `RequestManager#markRequest`.
2. The manager retrieves the request and rejects an already fulfilled request.
3. `Inventory#deductTotalAvailableQuantity` checks total available stock before any deduction.
4. Inventory deducts across categories in display order and removes depleted entries.
5. Only after successful deduction does the manager mark the request fulfilled and print confirmation.

Insufficient stock changes neither inventory nor status. These guarantees cover expected validation failures in the single-threaded application, without recovery from process termination. Dayna's storage implementation persists requests across restarts.

Design trade-offs: Cross-category deduction matches requests, which have no category. Display-order deduction is deterministic. Final fulfillment prevents repeated deductions and avoids deciding which categories receive restored units. Injecting a clock makes today/past-date boundaries reproducible in tests.

Verification: Inventory item, manager, and availability tests cover validation, quantities, duplicates, case-insensitive totals, overflow, and deductions. Request model, manager, parser, and logging tests cover ordering, fixed-clock boundaries, live stock, successful and rejected fulfillment, and final status through manager and command paths. See the team's UI test plan.

Limitations: Requests have no persistent identifiers, cannot be edited in place, and remain after their deadlines. Refresh the list before using an index. Fulfillment creates no loan and supports no undo; deleting a fulfilled request does not replenish stock.